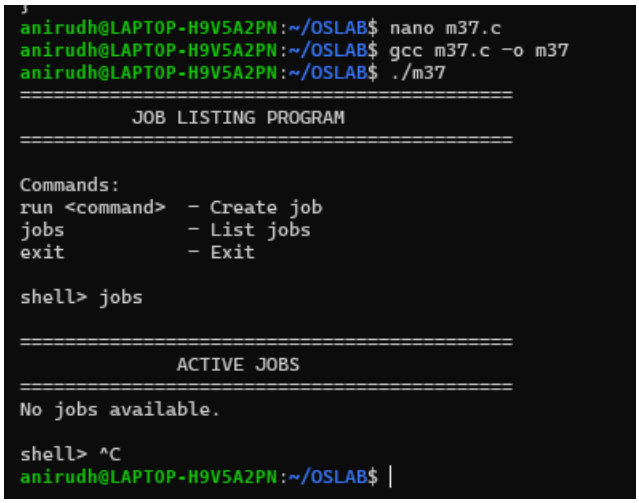

anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m37.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m37.c -o m37
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m37
==========
JOB LISTING PROGRAM
==========
Commands:
run <command> - Create job
jobs - List jobs
exit - Exit
shell> jobs
==========
ACTIVE JOBS
==========
No jobs available.
shell> ^C
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$

```
aniruddh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m37.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define MAX_COMMAND 256

typedef enum
{
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

const char *get_state(JobState state)
{
    switch (state)
    {
        case RUNNING:
            return "Running";

        case STOPPED:
            return "Stopped";

        case COMPLETED:
            return "Completed";
    }

    return "Unknown";
}

void update_states()
{
    int i;
    int status;

    for (i = 0; i < job_count; i++)
    {
        if (jobs[i].state == COMPLETED)
            continue;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED
        );

        if (result == 0)
            continue;

        if (result == jobs[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                jobs[i].state = STOPPED;
            }
            else if (WIFEXITED(status) ||
                     WIFSIGNALED(status))
            {
```

```
            }
        }
    }
}

void add_job(
    pid_t pid,
    const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Maximum job limit reached.\n");
        return;
    }

    jobs[job_count].job_id =
        job_count + 1;

    jobs[job_count].pid =
        pid;

    jobs[job_count].pgid =
        pid;

    strcpy(
        jobs[job_count].command,
        command
    );

    jobs[job_count].state =
        RUNNING;

    printf(
        "[%d] %d started: %s\n",
        jobs[job_count].job_id,
        pid,
        command
    );

    job_count++;
}

void list_jobs()
{
    int i;

    update_states();

    printf("\n");
    printf("==========================================\n");
    printf("              ACTIVE JOBS\n");
    printf("==========================================\n");

    if (job_count == 0)
    {
        printf("No jobs available.\n");
        return;
    }

    printf(
        "%-5s %-8s %-8s %-12s %s\n",
        "ID",
        "PID",
        "PGID",
        "STATE",
        "COMMAND"
    );

    printf(
        "------------------------------------------------------------\n"
    );

    for (i = 0; i < job_count; i++)
    {
        printf(
            "%-5d %-8d %-8d %-12s %s\n",
            jobs[i].job_id,
```

```
void launch_job(char *command)
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        char *args[32];
        int argc = 0;

        char *token =
            strtok(command, " \t\n");

        while (token != NULL &&
               argc < 31)
        {
            args[argc++] = token;
            token = strtok(NULL, " \t\n");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(EXIT_SUCCESS);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    setpgid(pid, pid);

    add_job(pid, command);
}

int main()
{
    char input[MAX_COMMAND];

    printf("============================================\n");
    printf("          JOB LISTING PROGRAM\n");
    printf("============================================\n");

    printf("\nCommands:\n");
    printf("run <command>  - Create job\n");
    printf("jobs           - List jobs\n");
    printf("exit           - Exit\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;
```

```
    char input[MAX_COMMAND];

    printf("============================================\n");
    printf("          JOB LISTING PROGRAM\n");
    printf("============================================\n");

    printf("\nCommands:\n");
    printf("run <command>  - Create job\n");
    printf("jobs           - List jobs\n");
    printf("exit           - Exit\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
            continue;
        }

        if (strncmp(input, "run ", 4) == 0)
        {
            char command[MAX_COMMAND];

            strcpy(
                command,
                input + 4
            );

            launch_job(command);
            continue;
        }

        printf("Invalid command.\n");
    }

    for (int i = 0;
         i < job_count;
         i++)
    {
        if (jobs[i].state != COMPLETED)
        {
            waitpid(
                jobs[i].pid,
                NULL,
                0
            );
        }
    }

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_COMMAND 256

pid_t shell_pgid;

void launch_foreground(char *command)
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /*
         * Child creates its own process group.
         */
        setpgid(0, 0);

        /*
         * Restore default signal handling.
         */
        signal(SIGINT, SIG_DFL);
        signal(SIGTSTP, SIG_DFL);
        signal(SIGQUIT, SIG_DFL);

        /*
         * Transfer terminal control to child.
         */
        tcsetpgrp(
            STDIN_FILENO,
            getpid()
        );

        char *args[32];
        int argc = 0;

        char *token =
            strtok(command, " \t\n");

        while (token != NULL &&
               argc < 31)
        {
            args[argc++] = token;
            token = strtok(NULL, " \t\n");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(EXIT_SUCCESS);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    /*
     * Parent places child in its process group.
     */
    setpgid(pid, pid);

    /*
     * Give terminal to child.
     */
    tcsetpgrp(
        STDIN_FILENO,
        pid
    );

    /*
     * Wait for foreground job.
     */
    waitpid(
        pid,
        &status,
        WUNTRACED
    );

    /*
     * Take terminal back.
     */
    tcsetpgrp(
        STDIN_FILENO,
        shell_pgid
    );

    if (WIFSTOPPED(status))
    {
        printf(
            "\nJob stopped. PID = %d\n",
            pid
```

```
        );
    }
    else if (WIFEXITED(status))
    {
        printf(
            "\nJob completed. Exit status = %d\n",
            WEXITSTATUS(status)
        );
    }
    else if (WIFSIGNALED(status))
    {
        printf(
            "\nJob terminated by signal.\n"
        );
    }
}

void resume_job(pid_t pid)
{
    int status;

    /*
     * Continue stopped process.
     */
    if (kill(
            -pid,
            SIGCONT) == -1)
    {
        perror("SIGCONT");
        return;
    }

    /*
     * Give terminal to process group.
     */
    if (tcsetpgrp(
            STDIN_FILENO,
            pid) == -1)
    {
        perror("tcsetpgrp");
        return;
    }

    printf(
        "Job %d resumed in foreground.\n",
        pid
    );

    /*
     * Wait for completion or stop.
     */
    if (waitpid(
            pid,
            &status,
            WUNTRACED) == -1)
    {
        perror("waitpid");
    }

    /*
     * Restore terminal to shell.
     */
    tcsetpgrp(
        STDIN_FILENO,
        shell_pgid
    );

    if (WIFSTOPPED(status))
    {
        printf(
            "Job stopped again.\n"
        );
    }
    else if (WIFEXITED(status))
    {
        printf(
            "Job completed.\n"
        );
    }
}

int main()
{
    char input[MAX_COMMAND];

    shell_pgid =
        getpid();

    /*
     * Put shell in its own process group.
     */
    setpgid(
        shell_pgid,
        shell_pgid
    );

    /*
     * Give terminal to shell.
     */
    tcsetpgrp(
        STDIN_FILENO,
        shell_pgid
    );

    /*
     * Ignore interactive signals in shell.
     */
    signal(SIGINT, SIG_IGN);
```

```
    printf("========================================\n");
    printf("        FOREGROUND JOB CONTROL\n");
    printf("========================================\n");

    printf("\nCommands:\n");
    printf("run <command>       Run foreground job\n");
    printf("resume <PID>        Resume stopped job\n");
    printf("exit                Exit\n");

    while (1)
    {
        printf("\nfg-shell> ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin) == NULL)
        {
            break;
        }

        input[strcspn(
            input,
            "\n"
        )] = '\0';

        if (strcmp(
                input,
                "exit") == 0)
        {
            break;
        }

        if (strncmp(
                input,
                "run ",
                4) == 0)
        {
            char command[MAX_COMMAND];

            strcpy(
                command,
                input + 4
            );

            launch_foreground(
                command
            );
        }
        else if (strncmp(
                    input,
                    "resume ",
                    7) == 0)
        {
            pid_t pid =
                (pid_t)atoi(
                    input + 7
                );

            if (pid <= 0)
            {
                printf(
                    "Invalid PID.\n"
                );
            }
            else
            {
                resume_job(pid);
            }
        }
        else
        {
            printf(
                "Invalid command.\n"
            );
        }
    }

    printf(
        "\nForeground job control completed.\n"
    );

    return 0;
}
```